CASE STUDY REPORT

Title: Comparative Case Study on MERN Stack vs Traditional Web Development

Project Title: CareerHub – Job and Internship Portal

Submitted by:

Name:-Renukunta Anusha

Roll no:-160124748088

Name:-Kaavali Debhora

Roll no:-160124748313

Date of Submission: 28/10/2026

Year/Sem/Sec: III Year / 5th sem/ CSM-2

Case Study 2: CareerHub – Job and Internship Portal

1. Introduction

The CareerHub Job and Internship Portal is a full stack web application designed to help students and job seekers find suitable job and internship opportunities. Searching for opportunities across multiple platforms and tracking submitted applications can be difficult and time-consuming. This project provides a centralized platform where users can view available jobs, submit applications, register their details, and view submitted application records.

Full stack development is important because it integrates frontend development, backend programming, database management, and API communication into a single application. CareerHub uses React.js to develop the user interface, Node.js and Express.js to process requests, and MongoDB to store user and application information.

The system can be used in job portals, college placement systems, company recruitment websites, internship platforms, and online

application tracking systems. It demonstrates how full stack technologies can be used to develop a dynamic and data-driven web application.

2. Methods Compared

This case study compares the MERN stack with traditional web development approaches.

The MERN stack consists of MongoDB, Express.js, React.js, and Node.js. It provides a JavaScript-based approach to full stack development. React.js is used to build the frontend interface, Express.js handles HTTP requests and REST APIs, Node.js executes backend code, and MongoDB stores application data in a NoSQL document format. Mongoose is used to define database schemas and interact with MongoDB.

Traditional web development uses separate technologies for the frontend, backend, and database. For example, HTML, CSS, and JavaScript can be used for the frontend, PHP or Java for the backend, and MySQL for relational data storage. This approach is suitable for many web applications but may require developers to work with multiple programming languages and integration methods.

Feature	MERN Stack	Traditional Web Development
Frontend	React.js	HTML, CSS, JavaScript, or other frameworks
Backend	Node.js and Express.js	PHP, Java, Python, or other technologies
Database	MongoDB (NoSQL)	Often MySQL or another relational database
Programming language	Primarily JavaScript	May use multiple languages
Data storage	Documents and collections	Often tables and rows
Communication	REST APIs	Depends on the architecture

The MERN stack was selected for CareerHub because it supports interactive user interfaces, efficient API communication, and persistent database storage. It also provides a consistent development approach by using JavaScript across the main application layers.

3. Architecture

CareerHub follows a client–server architecture in which the frontend communicates with the backend through REST APIs. The backend processes requests and interacts with MongoDB to store and retrieve information.

System Architecture:

User → React.js Frontend → Express.js REST API → Mongoose → MongoDB Database

3.1 Request–Response Cycle

The request–response cycle describes how information moves through the application.

· The user selects a job or submits an application through the React interface.

• React sends an HTTP request to the appropriate backend API endpoint.

• Express.js receives the request and validates the required information.

• Mongoose interacts with MongoDB to store or retrieve the requested data.

• The backend returns a JSON response indicating the result of the operation.

• React displays the response to the user, such as a successful application message or the list of submitted applications.

• For example, when a user applies for a job, the frontend sends a POST request to /api/applications. The backend validates the submitted information, saves the application in MongoDB, and returns a response to the frontend.

3.2 Authentication Flow

The application includes a user registration endpoint. The user submits their name, email address, and password through the registration form. The backend validates the required fields and checks whether the email address already exists. If the email is new, the password is hashed using bcrypt before the user details are stored in MongoDB. The server then returns a success or error response.

3.3 State Update Logic

When a user submits an application, the frontend sends the application details to the backend. After the backend stores the information in MongoDB, it returns a response to the frontend. The application records can then be retrieved and displayed on the Applications page.

The state update process follows this sequence:

User Action → API Request → Backend Processing → Database Operation → API Response → Frontend Update

4. Implementation

The CareerHub Job and Internship Portal was implemented using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for database management. Mongoose was used for database integration, and bcrypt was used to hash passwords during user registration.

1. src/App.jsx — Frontend

```
import { useEffect, useState } from "react";

const API = "http://localhost:5001/api";

function App() {
  const [jobs, setJobs] = useState([]);
  const [applications, setApplications] = useState([]);
  const [page, setPage] = useState("jobs");

  const [form, setForm] = useState({
    name: "",
    email: ""
  });

  useEffect(() => {
    fetch(`${API}/jobs`)
      .then(res => res.json())
      .then(data => setJobs(data))
      .catch(err => console.error(err));
  }, []);

  const applyJob = async (job) => {
```

```
  try {
    const res = await fetch(`${API}/applications`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        ...form,
        title: job.title,
        company: job.company,
        type: job.type
      })
    });

    const data = await res.json();
    alert(data.message);

    if (res.ok) {
      setForm({ name: "", email: "" });
    }
  } catch (error) {
    alert("Unable to submit application");
  }
};

const loadApplications = async () => {
  try {
    const res = await fetch(`${API}/applications`);
    const data = await res.json();
    setApplications(data);
    setPage("applications");
  } catch (error) {
    console.error(error);
  }
};

return (
  <div>
    <h1>CareerHub</h1>

    <button onClick={() => setPage("jobs")}>
      Jobs
    </button>
```

```
<button onClick={loadApplications}>
  Applications
</button>

{page === "jobs" && (
 <div>
   <h2>Available Jobs</h2>

   <input
     placeholder="Your Name"
     value={form.name}
     onChange={e =>
       setForm({ ...form, name: e.target.value })
     }
   />

   <input
     placeholder="Your Email"
     value={form.email}
     onChange={e =>
       setForm({ ...form, email: e.target.value })
     }
   />

   {jobs.map(job => (
     <div key={job.id}>
       <h3>{job.title}</h3>
       <p>{job.company}</p>
       <p>{job.type}</p>
       <button onClick={() => applyJob(job)}>
         Apply Now
       </button>
     </div>
   ))}
 </div>
)}

{page === "applications" && (
 <div>
   <h2>My Applications</h2>

   {applications.map(app => (
```

```
            <div key={app._id}>
              <h3>{app.title}</h3>
              <p>{app.company}</p>
              <p>{app.status}</p>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

export default App;
```

2. backend/server.js — Backend

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/careerhub")
  .then(() => console.log("MongoDB connected successfully"))
  .catch(err => console.log(err));

const applicationSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  title: { type: String, required: true },
  type: { type: String, required: true },
  company: { type: String, required: true },
  status: { type: String, default: "Pending" }
}, { timestamps: true });

const Application = mongoose.model(
  "Application",
  applicationSchema
);
```

```
const jobs = [
  {
    id: 1,
    title: "Python Developer",
    company: "TCS",
    type: "Full-time",
    location: "Hyderabad"
  },
  {
    id: 2,
    title: "AI/ML Intern",
    company: "Google",
    type: "Internship",
    location: "Bangalore"
  },
  {
    id: 3,
    title: "Web Developer",
    company: "Infosys",
    type: "Full-time",
    location: "Hyderabad"
  },
  {
    id: 4,
    title: "Data Science Intern",
    company: "Microsoft",
    type: "Internship",
    location: "Remote"
  }
];

app.get("/api/jobs", (req, res) => {
  res.json(jobs);
});

app.post("/api/applications", async (req, res) => {
  try {
    const { name, email, title, company } = req.body;
    const type = req.body.type || "Full-time";

    if (!name || !email || !title || !company) {
      return res.status(400).json({
        message: "Required fields are missing"
```

```
      });
    }

    const application = await Application.create({
      name,
      email,
      title,
      type,
      company,
      status: "Pending"
    });

    res.status(201).json({
      message: "Application submitted successfully!",
      application
    });
  } catch (error) {
    res.status(500).json({
      message: "Application submission failed"
    });
  }
});

app.get("/api/applications", async (req, res) => {
  try {
    const applications = await Application.find()
      .sort({ createdAt: -1 });

    res.json(applications);
  } catch (error) {
    res.status(500).json({
      message: "Failed to retrieve applications"
    });
  }
});

app.listen(5001, () => {
  console.log("CareerHub API running at http://localhost:5001");
});
```

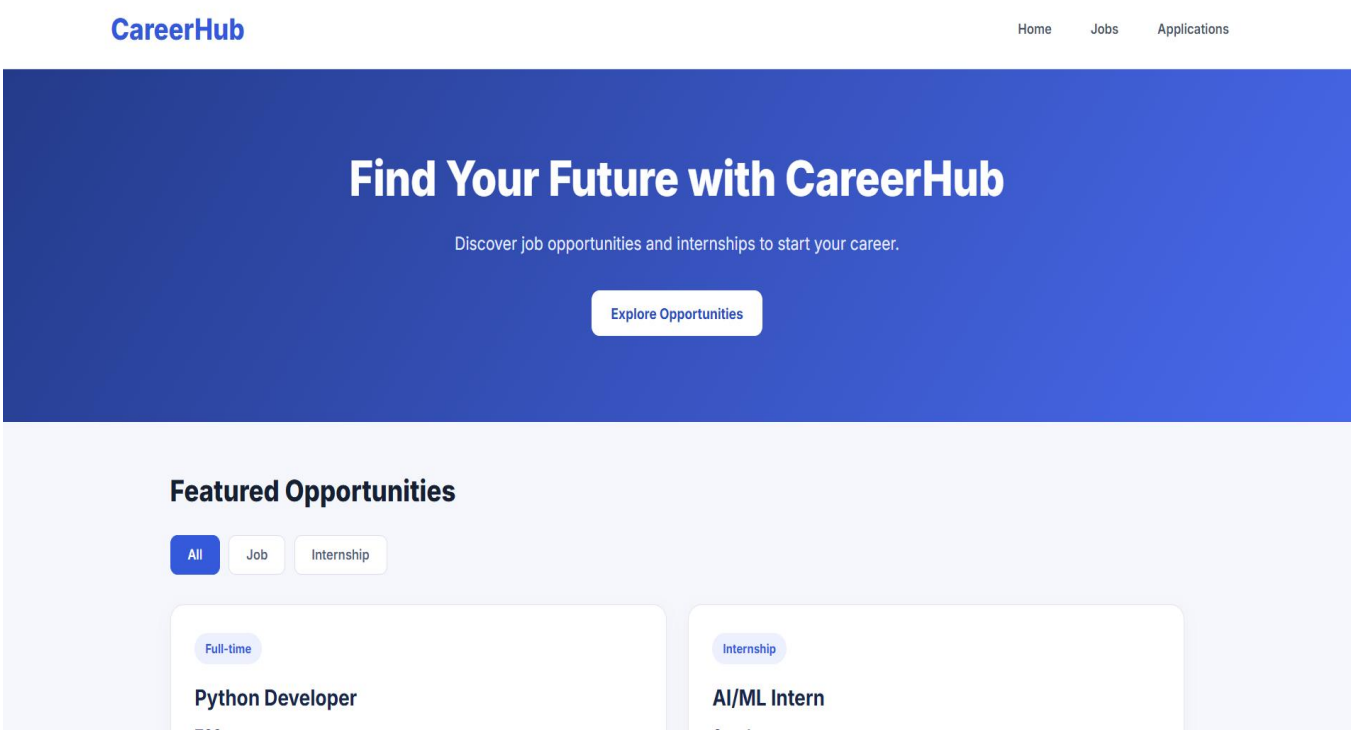
3. src/main.jsx — React Entry Point

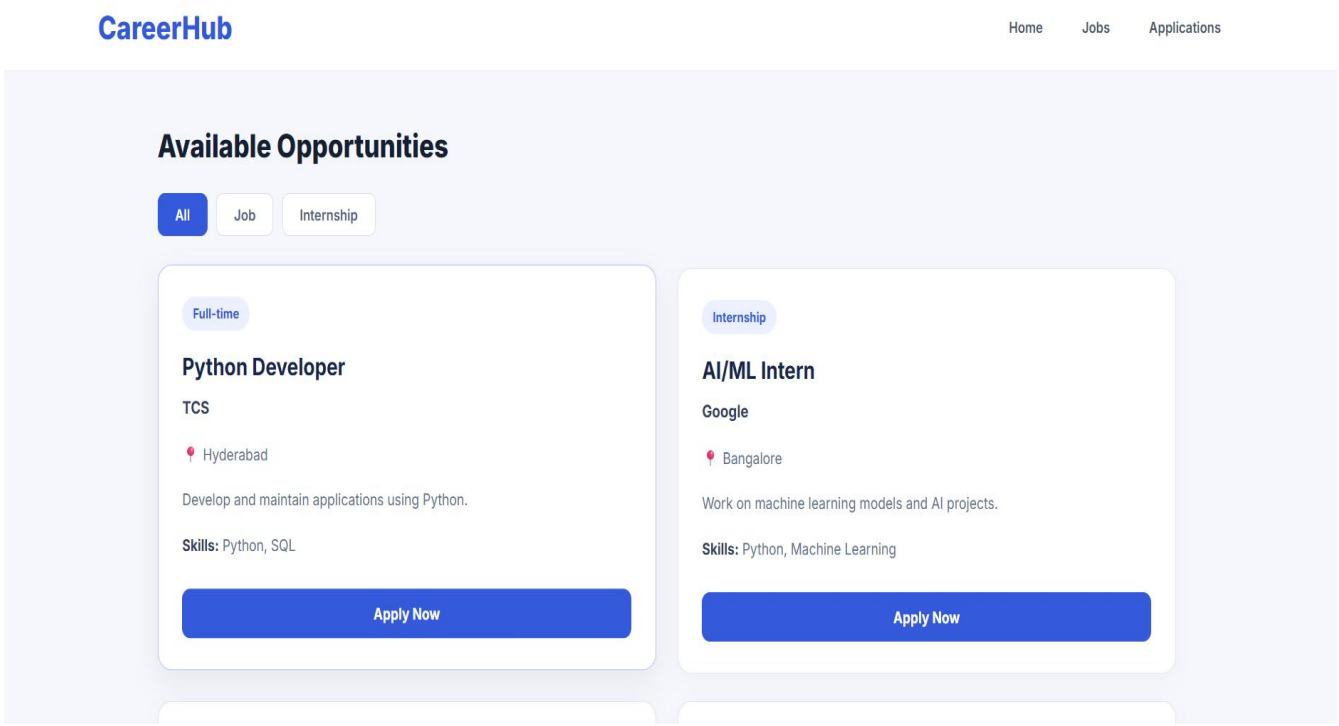
```
import { StrictMode } from "react";
```

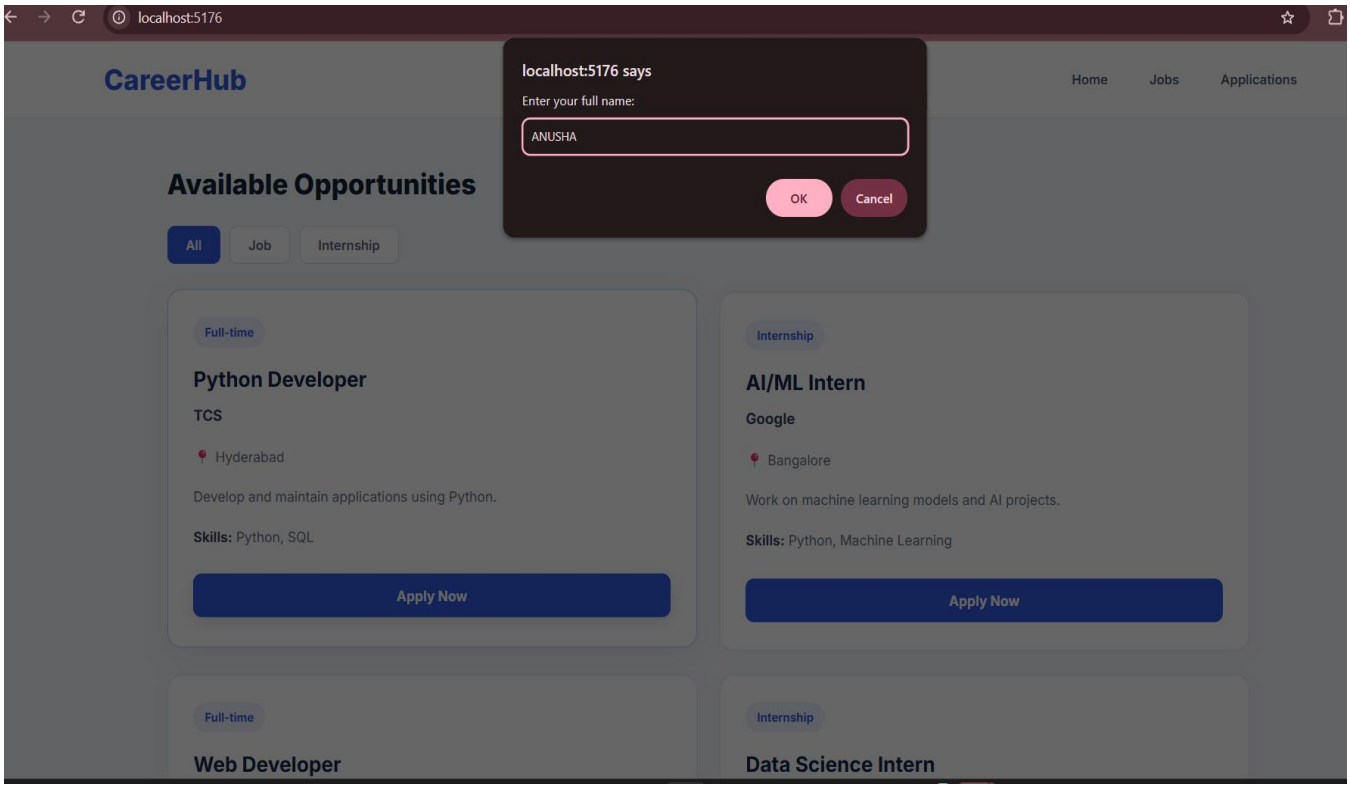
```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

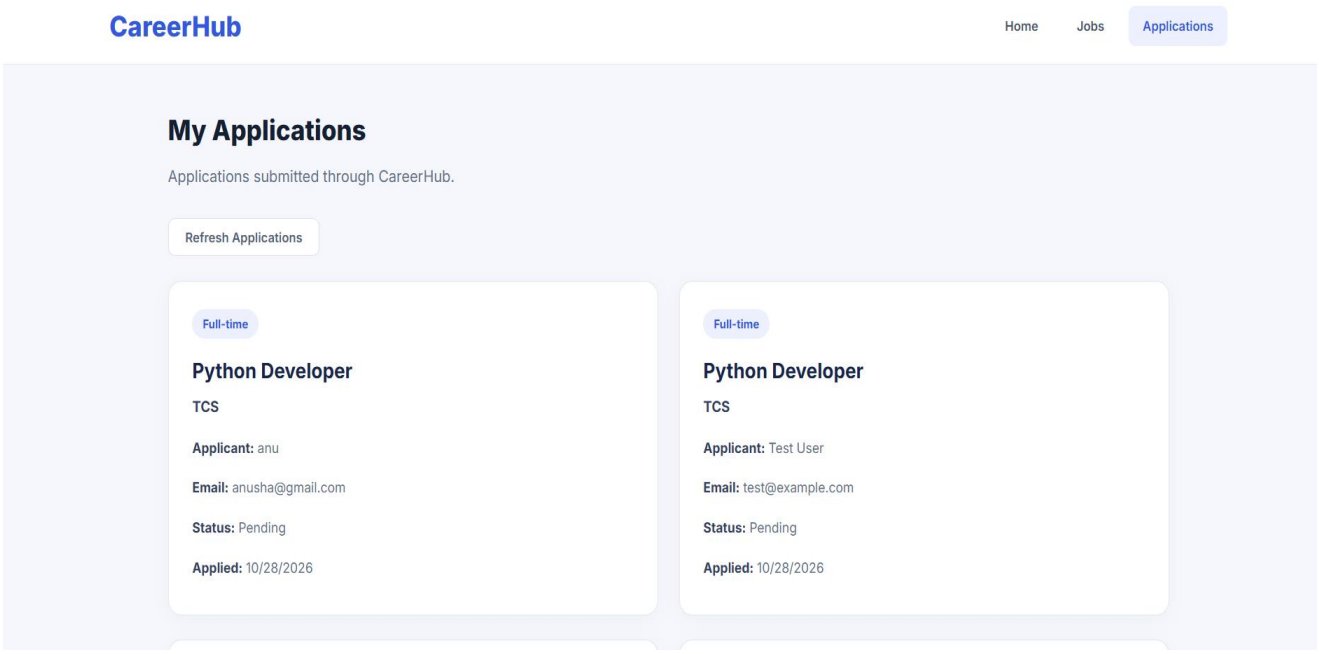
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

4. Results :-









6. Conclusion

The CareerHub Job and Internship Portal was developed using the MERN stack to display job opportunities, submit applications, and view application details. The project demonstrates frontend and backend integration, REST API communication, and MongoDB database connectivity. In the future, features such as resume uploads, advanced job search, and recruiter dashboards can be added to improve the system.